ASRP: Adaptive Support-Based Response Prediction for Test-Time AI Capability Evaluation

Yuhan Chiu
National Taiwan University

Abstract

Predicting whether an AI system will solve a particular evaluation item requires accounting for item difficulty, subject ability, and the small number of revealed labels. We present **ASRP (Adaptive Support-Based Response Prediction)**, a method for predicting subject–item response probabilities in the Predictive AI Evaluation Competition. ASRP combines a public-data subject and metadata prior, a MiniLM-based amortized Rasch predictor, and a support-conditioned kernel probe. A budget-dependent ensemble combines these branches, followed by conservative topic residual calibration; source-keyed guards are available when a public-source identifier matches. Its streaming acquisition policy uses an adaptive covariance-gain rule with a bounded warm-up phase. We evaluate 92 public-source groups over three fixed seeds, with 22,357 evaluation outcomes per seed. A paired comparison of ASRP and Empirical Mean under both Random and Adaptive acquisition separates predictor quality from acquisition effects. ASRP + Adaptive achieves a mean Brier ALC of 0.1987, compared with 0.2008 for ASRP + Random and 0.2435 for Empirical Mean + Random (lower is better).

1 Introduction

Modern AI evaluations produce response tables with a binary outcome for each subject–item pair. A benchmark average is useful for ranking systems, but it does not provide a probability for a new item and it can be unstable when only a small subset of items is queried. The Predictive AI Evaluation Competition (PAIEC) therefore asks participants to predict the probability that a subject answers each item correctly while revealing at most 31 labels per subject–benchmark pair.

Our goal is a predictor that uses item text and visible metadata at label budget zero, then adapts to revealed labels without overreacting to a small or unrepresentative support set. We implement **ASRP** as a competition submission rather than as a new psychometric theory.

Relation to item-response modeling. Lord's treatment of item response theory (IRT) models the probability of a correct response as a function of test-taker ability and item difficulty [Lord, 1980]. This is the basis for ASRP's Rasch-style probability: a public subject prior supplies the initial ability, the frozen text model supplies item difficulty, and revealed labels update ability with a penalized Newton–Raphson step.

Content-based and adaptive evaluation. Reliable and Efficient Amortized Model-based Evaluation shows that benchmark averages can be distorted by question difficulty, and proposes amortizing difficulty from question content together with adaptive item selection [Truong et al., 2025]. ASRP uses the same two ideas in the competition setting: MiniLM features predict unseen-item difficulty, while `adaptive_4_warm` implements support selection when the remaining budget permits discretionary choices. ASRP does not use that paper's question generator or evaluation model.

Preprint.

Task-level capability prediction. Agent Psychometrics extends IRT to predict task-level success for agentic coding benchmarks using task features and separate ability factors [Ge et al., 2026]. ASRP follows the same subject–item perspective, but uses the competition's visible metadata, binary outcomes, and a support-conditioned kernel residual instead of an agent/scaffold decomposition.

Capability calibration. *On Calibration of Large Language Models: From Response to Capability* distinguishes confidence that one sampled response is correct from confidence that a model can solve a query overall [Yang et al., 2026]. ASRP targets the latter quantity: $p(x)$ is the probability that the subject answers the current item correctly, and Brier score evaluates this query-level probability. This motivates our query-level prediction target; ASRP uses its own topic residual correction rather than reproducing that paper's confidence estimators.

ASRP combines frozen text-based difficulty and public subject priors with a kernel residual fitted to acquired labels. Fixed budget-specific weights blend these signals, followed by shrunk topic corrections.

We compare against the official Empirical Mean on a local public proxy. Competition-specific artifacts use permitted public measurement-db data; runtime inference never queries hidden responses.

2 Competition setting

At runtime the platform supplies a pair of dictionaries $x = (s, i)$ describing a subject and an item, together with a list of previously acquired binary observations. The predictor returns a finite probability in $[0, 1]$. Eligible subject–benchmark groups have at least 80 distinct items. The platform creates a fixed acquisition/evaluation split and evaluates the same evaluation items at the **six label budgets**

$$n \in \{0, 1, 3, 7, 15, 31\}.$$

Acquisition items are never scored and evaluation outcomes are never revealed. The optional acquisition hook receives the current candidate, acquired labels, and remaining-label context; the interface can also supply a current prediction when requested. ASRP uses the first three inputs.

For a budget n and a subject–benchmark group with N_{eval} evaluated responses, the Brier score is the mean squared error between the predicted probability p and the binary outcome y:

$$B_n = \frac{1}{N_{\text{eval}}} \sum_{j=1}^{N_{\text{eval}}} (p_j - y_j)^2.$$

The group-level Brier ALC is the normalized area on the logarithmic budget axis and is equivalent to

$$\text{ALC} = 0.1B_0 + 0.2B_1 + 0.2B_3 + 0.2B_7 + 0.2B_{15} + 0.1B_{31}.$$

The official score averages these group-level ALC values equally across subject–benchmark groups. **Lower is better.** All reported local tables use this same weighting.

3 Data and permitted training

The predictor uses frozen artifacts trained and calibrated from the public `aims-foundations/measurement-db` release [AIMS Foundations, 2026]. The evaluation starts from a fixed inventory of 142 binary sources and retains sources containing a complete subject group with 80–1,000 distinct binary-response items. This yields **92 sources**; 50 sources have no qualifying group. One eligible subject is fixed per source, and all items in that group are retained without truncation. The selected groups contain 44,741 items in total. Appendix B lists the evaluated sources; the manifest records both inclusions and exclusions. Upstream metadata and licenses remain authoritative.

The subject prior and metadata statistics are frozen public-response aggregates. Their runtime use and fallback rules are specified in Appendix A; reproducing the reported scores uses the released weights without refitting them. MiniLM is used as a pretrained encoder; the packaged encoder is `sentence-transformers/all-MiniLM-L6-v2`, based on MiniLM [Wang et al., 2020]. Its weights are **frozen at runtime**. The difficulty model, priors, kernel configuration, calibration statistics, and cluster centroids are prepared offline and bundled with the submission; kernel residual

coefficients are fitted online from revealed labels. The current submission uses **no external LLM or web search API during scoring**. At runtime, the program uses only the supplied subject and item attributes, anonymous benchmark identifiers, and labels revealed within the allowed budget; it does not access hidden evaluation outcomes.

4 ASRP method

For a query subject s and item i in benchmark b, let $\mathcal{D}_b$ be the revealed support and $\mathcal{D}_s$ its rows for the current subject. Write $n = |\mathcal{D}_s|$ and $S_s = \sum_{r \in \mathcal{D}_s} y_r$. All predictions below concern this query; their arguments are suppressed when unambiguous. Figure 1 shows the computation. Appendix A gives the shared notation, numerical details, and corresponding code functions.

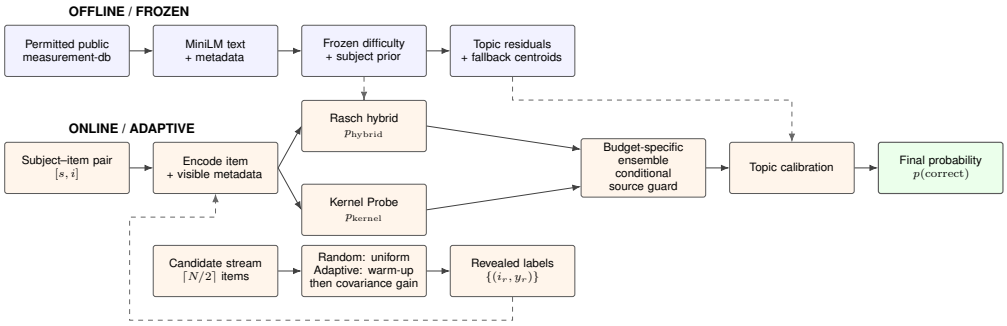


Figure 1: ASRP overview. Frozen public-data artifacts supply difficulty, prior, and calibration estimates. Revealed labels adapt the Rasch hybrid and kernel probe before budget-specific ensembling and topic correction. The source guard is inactive for anonymous proxy identifiers. For a complete group of N items, the candidate pool contains $\lceil N/2 \rceil$ items. Random samples uniformly; Adaptive skips up to 16 warm-up candidates before covariance-gain selection. Both supply nested label prefixes.

4.1 Item representation and amortized difficulty

MiniLM encodes the normalized item text as $e_i \in \mathbb{R}^{384}$. Six deterministic numeric text features form $z_i \in \mathbb{R}^6$. A frozen feed-forward network standardizes their concatenation and predicts a clipped difficulty:

$$d_i = \text{clip}(f_\phi([e_i; z_i]), -2, 2)\,.$$

The network has dimensions 390–128–32–1 and ReLU hidden layers. This amortized estimate supplies a difficulty for an unseen item without fitting an item-specific parameter online. Appendix A.1 specifies text limits, pooling, and standardization.

4.2 Public prior and Rasch update

A frozen subject offset o_s, looked up by normalized model name, defines a prior mean and initial ability:

$$\mu_s = \text{clip}(0.5 + 0.5o_s, 0.05, 0.95), \qquad \theta_{0,s} = \text{logit}(\mu_s).$$

An unknown name has $o_s = 0$. With revealed labels, clipped Newton–Raphson iterations approximate the **maximum a posteriori (MAP) estimate** under a Bernoulli Rasch likelihood and Gaussian ability prior $\mathcal{N}(\theta_{0,s}, v_\theta)$, with $v_\theta = 2$. The resulting ability $\hat{\theta}_s$ gives

$$p_{\text{Rasch}} = \sigma(\hat{\theta}_s - d_i), \qquad \sigma(t) = (1 + e^{-t})^{-1}.$$

At $n = 0$, $\hat{\theta}_s = \theta_{0,s}$. Separately, a query-specific metadata prior $p_{\text{meta}}(s, i)$ combines frozen subject and visible-feature counts. The hybrid smooths observed successes toward this prior and blends the result with the Rasch probability:

$$p_{\text{smooth}} = \frac{S_s + \kappa p_{\text{meta}}(s,i)}{n + \kappa}, \qquad p_{\text{hybrid}} = (1 - \eta_n)p_{\text{smooth}} + \eta_n p_{\text{Rasch}}, \qquad \kappa = 4.$$

Here η_n is the frozen Rasch mixing weight. Appendix A.2 gives the prior lookup and Newton updates; Appendix A.4 lists the six mixing weights.

4.3 Support-conditioned kernel probe

The kernel branch adds an item-dependent correction to a conservative subject baseline. It encodes visible text, metadata, and interactor descriptions into semantic vectors $\mathbf{h}_i$ and lexical vectors $\boldsymbol{\ell}_i$. A locally normalized similarity kernel K is fitted using only the support features in $\mathcal{D}_b$; the query does not change this geometry.

For each supported subject s, let n_s and S_s be its revealed-label and success counts. A two-component Beta mixture has concentrations $(c_1, c_2) = (1, 8)$, prior masses $(\pi_1, \pi_2) = (0.3, 0.7)$, and posterior weights ω_{sk}. Its mean is

$$b_s = \sum_{k=1}^{2} \omega_{sk} \frac{c_k \mu_s + S_s}{c_k + n_s}.$$

Subject-centered kernel ridge regression, with penalty $\lambda_K = 0.3$, fits residual coefficients γ from the revealed labels. With query-to-support similarities $\mathbf{k}_i$ and subject mean fitted contribution $\bar{f}_s$, the prediction is

$$p_{\text{kernel}} = \text{clip}\big(b_s + \mathbf{k}_i^\top \gamma - \bar{f}_s, 0, 1\big)\,.$$

Appendix A.3 defines K, the Beta weight update, and the centered linear solve. If the current subject has no revealed labels, this branch returns μ_s. The implementation supports multiple subjects within a benchmark; this proxy uses one subject per group.

The ensemble gives the kernel weights $w_n = (0, 0, 0.2, 0.4, 1, 1)$ at budgets $(0, 1, 3, 7, 15, 31)$. Thus the Rasch hybrid supplies the cold-start prediction, while a larger support set allows the local residual to receive more weight. These are design choices encoded in the released configuration.

4.4 Topic calibration and source guard

A frozen topic correction $\Delta_n(i)$ adjusts the ensemble using the first matching topic entry in visible metadata. Without a match, the model uses the closest of eight frozen embedding centroids. Count-based shrinkage and caps limit the adjustment to 0.06 for explicit topics or 0.03 for clusters. Missing statistics give zero correction.

A benchmark-keyed guard $g_n(b) \in [0, 0.35]$ reduces kernel weight at $n \geq 15$ when a stored source key matches; it is zero otherwise. The final prediction is

$$\widetilde{w}_n(b) = w_n[1 - g_n(b)], \qquad p_{\text{final}} = \text{clip}([1 - \widetilde{w}_n(b)]p_{\text{hybrid}} + \widetilde{w}_n(b)p_{\text{kernel}} + \Delta_n(i), 0, 1)\,.$$

The guard is inactive for the anonymous identifiers in this proxy; topic correction remains active. Appendix A.4 gives the exact correction and lookup rules.

4.5 Adaptive acquisition

The `adaptive_4_warm` hook skips up to 16 early candidates to establish feature geometry, without revealing their labels. A reference set $\mathcal{P}$ deduplicates the last 64 seen candidates by visible text, metadata, and interactor features. For the current candidate $*$, a conditional covariance gain G_* measures its potential reduction of uncertainty over $\mathcal{P}$. Normalizing and capping this gain, then raising it to power four, gives utility V_*.

For L labels and R candidates remaining, including the current candidate, the acquisition probability is

$$\pi_*^{\text{acq}} = \min\left(1, \frac{L}{R}\,\text{clip}\left(\frac{V_*}{\max(\overline{V}_{\mathcal{P}}, 10^{-12})}, 0.05, 20\right)\right), \qquad \overline{V}_{\mathcal{P}} = \frac{1}{|\mathcal{P}|}\sum_{j \in \mathcal{P}} V_j.$$

A deterministic uniform hash supplies the draw. When $R \leq L$ and $L > 0$, every remaining candidate is acquired, overriding warm-up. Appendix A.5 defines G_j, V_j, and the boundary cases. The rule uses only acquired support and already-seen/current features; it never accesses future candidates or unseen outcomes.

5 Experimental setup

5.1 Public proxy protocol

Complete groups and repeated splits. The local protocol retains complete groups with $80 \leq N \leq 1{,}000$ distinct items; the upper bound is a local eligibility filter, not truncation. Within each eligible source, SHA256 ordering with the fixed group-selection seed 20261028 selects one subject, identically across all experiments. The actual group sizes range from 90 to 1000. For repeated responses to an item, the smallest trial number and then response ID select one binary outcome, held fixed across policies and seeds. Splitting uses seeds **42, 43, and 44**: a seeded SHA256 ordering assigns the first $\lceil N/2 \rceil$ items to acquisition and the rest to evaluation. Thus candidate pools contain 45–500 items, and each seed scores **22,357 evaluation outcomes**. The dataset revision, table checksums, subject IDs, response IDs, and three splits are fixed in the released manifest.

Paired predictor and acquisition comparison. We evaluate **ASRP and Empirical Mean under both Random and Adaptive acquisition**. Within a seed, both policies see the same candidate order and evaluation pool. Random acquisition accepts the current candidate with probability L/R, where L is the remaining label budget and R counts remaining candidates including the current one. Its random variates are deterministic hashes of the visible input and context, with seed-specific anonymous subject aliases. Adaptive acquisition executes the packaged `adaptive_4_warm` hook. Its warm-up is $\min(16, A - 31)$ for a candidate pool of size A; remaining candidates are mandatory when their count equals the remaining budget. Both predictors receive identical acquired labels *within each policy*, using nested prefixes at budgets $0, 1, 3, 7, 15, 31$. The two policies can select different labels. The acquisition context targets 31 labels throughout; checkpoints do not restart selection. Every checkpoint scores the same evaluation items within that seed.

Evaluation scope. Evaluation outcomes are never supplied online, but this acquisition/evaluation split is not independent of all offline fitting: frozen public training and calibration use the same release. The local scores may therefore be optimistic. They are not Codabench scores; private data and formative random draws can produce different results.

5.2 Baseline and metrics

We use the **Empirical Mean predictor** from the official PAIEC repository [Truong et al., 2026]. For n acquired binary labels in the current subject–benchmark group, it predicts

$$p_{\mathrm{EM}} = \begin{cases} 0.5, & n = 0, \\ \frac{1}{n} \sum_{k=1}^{n} y_k, & n > 0. \end{cases}$$

It uses neither item features nor a learned prior. We run this unchanged predictor on each policy's acquired evidence. Empirical Mean + Random corresponds to its default acquisition choice; Empirical Mean + Adaptive isolates the effect of giving the same predictor ASRP's selected labels. The local Random policy reproduces uniform sampling without replacement, not the private server's particular random draw.

For each seed and policy, we compute Brier scores within each subject–source group and average groups equally, then average the three seed-level scores equally. Groups have different numbers of evaluation items, so pooling all response rows would give different weights and is not used. Brier ALC uses the weights in Section 2. We report the individual seed ALCs and their sample standard deviation; these describe variation across the three specified splits, not a confidence interval over all possible evaluations.

6 Results and analysis

6.1 Public proxy results

Table 1 reports the **four predictor–acquisition combinations**, averaged equally over groups and then over seeds 42, 43, and 44. ASRP + Adaptive has a mean Brier ALC of 0.1987, compared with 0.2435 for Empirical Mean + Random (an absolute reduction of 0.0447). Its aggregate Brier score is

lower at 6 of the six budgets. The two predictors share identical labels within each acquisition policy; the Random and Adaptive policies use identical candidate orders and evaluation pools.

Predictor + acquisition	B_0	B_1	B_3	B_7	B_{15}	B_{31}	ALC
Empirical Mean + Random	0.2500	0.3488	0.2518	0.2086	0.1910	0.1842	0.2435
Empirical Mean + Adaptive	0.2500	0.3594	0.2366	0.2003	0.1883	0.1837	0.2403
ASRP + Random	0.2263	0.2166	0.2075	0.1947	0.1845	0.1755	0.2008
ASRP + Adaptive	0.2263	0.2161	0.2043	0.1912	0.1819	0.1738	0.1987

Table 1: Public proxy Brier scores averaged equally over 92 groups and seeds 42, 43, and 44. Lower is better. Evaluation pools are matched across policies; acquired labels are matched across predictors within each policy.

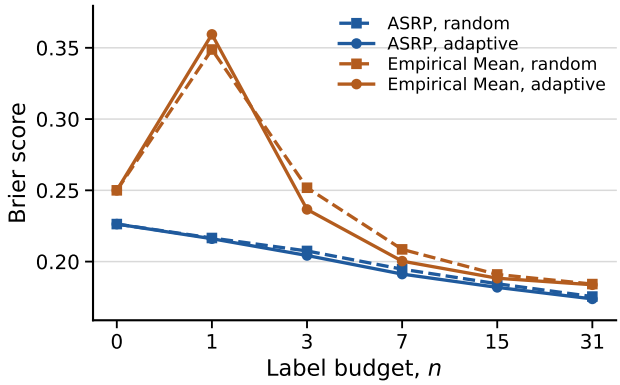


Figure 2: Mean public-proxy Brier curves for the four predictor–acquisition combinations on the official $\log_2(1+n)$ budget axis. Solid lines denote Adaptive acquisition and dashed lines Random; values match Table 1.

Predictor + acquisition	Seed 42	Seed 43	Seed 44	SD
Empirical Mean + Random	0.2467	0.2401	0.2435	0.0033
Empirical Mean + Adaptive	0.2386	0.2427	0.2396	0.0021
ASRP + Random	0.2020	0.1984	0.2020	0.0021
ASRP + Adaptive	0.1983	0.1983	0.1995	0.0007

Table 2: Group-averaged Brier ALC for each split/acquisition seed. SD is the sample standard deviation over the three seed scores. Lower ALC is better.

6.2 Prediction and acquisition effects

Predictor effect with acquisition held fixed. Under Random acquisition, ASRP's mean ALC is 0.2008 versus 0.2435 for Empirical Mean, a reduction of 0.0426. Under Adaptive acquisition, ASRP's mean ALC is 0.1987 versus 0.2403 for Empirical Mean, a reduction of 0.0416. Empirical Mean + Random rises from 0.2500 at zero labels to 0.3488 at one label: a single outcome makes its prediction either zero or one. ASRP smooths early evidence using public priors. At budget zero the acquisition policies have no effect. The Rasch hybrid followed by topic correction supplies the initial prediction; the kernel branch receives increasing weight as labels become available.

Acquisition effect with the predictor held fixed. For ASRP, Adaptive lowers mean ALC by 0.0021 relative to Random, and has lower ALC in all three seeds. For Empirical Mean, Adaptive lowers mean ALC by 0.0031 relative to Random, and has lower ALC in 2 of the three seeds. The closest ASRP pair differs by only -0.00005 (Adaptive minus Random, seed 43). Table 2 shows the

individual seed results and their sample standard deviation; three seeds summarize sensitivity to these draws rather than establishing universal superiority.

Selection capacity. Across the full experiment, 273 of 276 Adaptive trajectories execute at least one discretionary decision. The 3 trajectories without discretionary choices belong to one 90-item group: 14 warm-up skips leave exactly 31 candidates, all mandatory. Topic calibration is active; the source-name guard remains inactive because the benchmark aliases are anonymous.

7 Limitations

Sparse metadata and local adaptation. Some public items lack usable topic metadata, motivating the frozen embedding-cluster fallback. Topic groups with few observations can have residual estimates dominated by sampling noise. This motivates shrinking and capping topic corrections. Likewise, an unrepresentative acquired support set can cause the kernel to over-update. The source guard only limits its weight when a public-source key matches; anonymous or unseen sources do not receive this adjustment. These considerations motivate conservative correction magnitudes; neither topic calibration nor local adaptation guarantees improvement on every source.

8 Reproducibility and disclosure

The release separates inference code in `src/asrp` from frozen artifacts in `artifacts/asrp`. The entry points are `model.py` and `labeling.py`. From the repository root, run:

```
uv sync --locked
uv run hf auth login
uv run --locked python tools/reproduce.py
```

After accepting upstream access conditions and authenticating with a Hugging Face read token, this command downloads and verifies the pinned tables, reconstructs all splits, builds the submission, and evaluates all four combinations at three seeds. It checks every prediction against the published run within 10^{-7} and requires identical acquired labels. Frozen weights are used directly without refitting or parameter selection. The CPU evaluator uses four workers and caches only deterministic item features. Two independent complete executions produced identical predictions and acquisition records.

`experiments/` records the protocol, checksums, and selected subject/item/response IDs. `results/proxy` contains predictions, acquired labels, decisions, and metrics. `tools/update_report_results.py` generates the reported numbers; task text is downloaded from upstream.

Kernel weights are in `src/asrp/model.py`; Rasch settings and calibration are in `artifacts/asrp/runtime_config.json` and `artifacts/asrp/topic_calibration.json`. Code, frozen weights, the locked CPU environment, and report source are available at `https://github.com/Hubert0916/arsp`. The official checker remains in the upstream starting kit.

The competition-specific artifacts were produced from the permitted public measurement-db inventory. We use the pretrained MiniLM encoder and cite its model family. Codex was used as a coding assistant during development; it is not used as an online predictor in the current submission. No API key is included in the codebase or report.

9 Conclusion

ASRP provides a practical, budget-aware predictor for subject–item capability evaluation. Its main engineering principle is to use content-based and psychometric structure at cold start, use support-conditioned residuals only when evidence is available, and shrink topic corrections and late-budget updates toward conservative defaults. The three-seed public proxy gives ASRP + Adaptive a mean Brier ALC of 0.1987. Compared with Random acquisition using the same predictor, Adaptive reduces mean ALC by 0.0021. The paired design separates changes in prediction quality from changes in which labels are acquired.

A Runtime details

These equations specify inference with the released frozen artifacts at the six competition budgets. They use the same symbols as Section 4. The reproduction command evaluates these artifacts without repeating their original offline fitting. Throughout, $\mathrm{clip}(x, l, u) = \min(u, \max(l, x))$, applied coordinatewise to vectors, and $\mathrm{norm}(x) = x/\max(\|x\|_2, 10^{-12})$.

Shared notation. A support row r contains a subject s_r, item i_r, and binary outcome y_r. The row collection $\mathcal{D}_b$ contains all revealed responses for benchmark b and has size M. Its subject-specific subset $\mathcal{D}_s$ has n_s rows and S_s successes. For the current query subject, the budget is $n = n_s \le 31$. In this proxy $M = n$; the formulas below also describe the implementation's multi-subject kernel. Runtime subject identity uses the complete subject dictionary, whereas frozen subject-prior lookup uses `normalized_name`.

We distinguish η_n (Rasch mixing weight), w_n (kernel mixing weight), and λ_K (ridge penalty). Beta component index $k \in \{1, 2\}$ has concentration c_k, prior mass π_k, and posterior mass ω_{sk}. Acquisition uses gain G_j, utility V_j, probability π_*^{acq}, and uniform variate ξ; none denotes an embedding coordinate.

A.1 Feature construction for Section 4.1

The Rasch encoder normalizes whitespace, takes the first 6,000 characters of item content, and tokenizes to at most 256 tokens. Masked mean pooling followed by L_2 normalization gives $e_i \in \mathbb{R}^{384}$. The six coordinates of z_i are $\log(1+\text{text length})/10$, digit fraction, non-ASCII fraction, and indicators for code, mathematical notation, and a question mark. Fractions and the question-mark indicator use the first 2,000 characters; code/math markers use the first 6,000. Text length is measured before these truncations.

For stored feature mean $\bar{\mathbf{x}}$ and scale $\boldsymbol{\sigma}_x$, the difficulty network receives $\mathrm{clip}(([e_i; z_i] - \bar{\mathbf{x}}) \oslash \boldsymbol{\sigma}_x, -10, 10)$, where $\oslash$ denotes coordinatewise division. Frozen weights in `difficulty_model.npz` implement the 390–128–32–1 map; the final scalar is clipped to $[-2, 2]$ by `_predict_hybrid`, after `_difficulty` evaluates the network.

The kernel encoder uses a separate text construction containing item content, metadata, and interactor descriptions. It pools up to four evenly spaced windows of 254 content tokens, each with special tokens, normalizes each pooled vector, and normalizes their sum to obtain $\mathbf{h}_i \in \mathbb{R}^{384}$. The lexical vector $\boldsymbol{\ell}_i \in \mathbb{R}^{8192}$ uses signed CRC32 hashing of lowercased tokens and adjacent token bigrams, weights $1 + \log(\text{count})$, and L_2 normalization. Only these semantic and lexical coordinates enter the kernel. See `ml_features.py`, `window_features.py`, and `evidence_features.py` under `src/asrp/`.

A.2 Priors and ability update for Section 4.2

Two distinct frozen priors. The scalar o_s comes from `binary_subject_prior.json`. It defines μ_s and $\theta_{0,s}$ exactly as in Section 4.2. The metadata prior $p_{\mathrm{meta}}(s, i)$ instead uses the success/count tables in `feature_prior.json`; it corresponds to `_feature_prior` and the runtime variable `p_source` in `src/asrp/model.py`. The released source-prior weight is one, prior shift is zero, and metadata-match adaptation weight is zero.

Let (A_0, N_0) be global public successes and counts, and (A_s, N_s) the subject totals. The frozen rates are

$$q_0 = \frac{A_0}{\max(N_0, 1)}, \qquad q_s = \frac{A_s + 8q_0}{N_s + 8}.$$

Unknown subjects use global totals. Their offset o_s is separately zero, so $\theta_{0,s} = 0$, but $p_{\mathrm{meta}}(s, i)$ need not equal 0.5. All released global totals have positive counts.

For each token t in the query metadata with stored global counts, let $q_t = A_t/\max(N_t, 1)$. When subject–token counts exist,

$$q_{st} = \frac{A_{st} + 8q_t}{N_{st} + 8}, \qquad \rho_{st} = \frac{\min(N_{st}, 80)}{\min(N_{st}, 80) + 12}.$$

Otherwise $q_{st} = q_t$ and $\rho_{st} = \min(N_t, 300)/[\min(N_t, 300) + 100]$. For the set $\mathcal{T}(i)$ of matching tokens,

$$p_{\text{meta}}(s, i) = \text{clip}\left(\frac{2q_s + \sum_{t\in\mathcal{T}(i)} \rho_{st} q_{st}}{2 + \sum_{t\in\mathcal{T}(i)} \rho_{st}}, 0.03, 0.97\right).$$

An empty token set gives clipped q_s. Token parsing, deduplication, and stop words follow `_feature_tokens`. The packaged artifact is present; if it were absent, the runtime fallback would be μ_s.

MAP objective and numerical update. Using only $\mathcal{D}_s$, the penalized Bernoulli log-likelihood is

$$\mathcal{L}_s(\theta) = \sum_{r\in\mathcal{D}_s} [y_r \log \sigma(\theta - d_{i_r}) + (1 - y_r) \log(1 - \sigma(\theta - d_{i_r}))] - \frac{(\theta - \theta_{0,s})^2}{2v_\theta}, \qquad v_\theta = 2.$$

Starting at $\theta_{0,s}$, `update_ability` computes

$$\begin{aligned} p_r(\theta) &= \sigma(\text{clip}(\theta - d_{i_r}, -40, 40)), \\ J_s(\theta) &= \sum_{r\in\mathcal{D}_s} (y_r - p_r(\theta)) - (\theta - \theta_{0,s})/v_\theta, \\ \mathcal{I}_s(\theta) &= \sum_{r\in\mathcal{D}_s} p_r(\theta)(1 - p_r(\theta)) + 1/v_\theta, \\ \delta_\theta &= \text{clip}\big(J_s(\theta)/\max(\mathcal{I}_s(\theta), 10^{-6}), -2, 2\big), \qquad \theta \leftarrow \theta + \delta_\theta. \end{aligned}$$

Here J_s is the gradient and $\mathcal{I}_s$ the positive negative-Hessian magnitude, not the Hessian itself. The routine stops after 30 iterations or when $|\delta_\theta| < 10^{-7}$, then clips $\hat{\theta}_s$ to $[-12, 12]$. With no labels it returns $\theta_{0,s}$ directly. The runtime sigmoid also clips its logit to $[-40, 40]$ when computing p_{Rasch}. The hybrid uses at most the first 31 matching labels in acquisition order.

A.3 Kernel calculations for Section 4.3

Support-only geometry and bandwidth. For $M > 1$ rows in $\mathcal{D}_b$, let $\bar{\mathbf{h}}$ and v_j^{feat} be the coordinatewise semantic mean and population variance (divisor M). Define semantic scales t_j and lexical weights ν_ℓ by

$$t_j = \left[v_j^{\text{feat}} + \max(\text{median}_j\, v_j^{\text{feat}}, 10^{-7})\right]^{-1/2}, \qquad \nu_\ell = \log\frac{1 + M}{1 + f_\ell} + 0.1,$$

where $f_\ell = \sum_{r\in\mathcal{D}_b} \mathbf{1}[|\ell_{i_r,\ell}| > 10^{-10}]$. Both support and query use

$$\widetilde{\mathbf{h}}_i = \text{norm}((\mathbf{h}_i - \bar{\mathbf{h}}) \odot \mathbf{t}), \qquad \widetilde{\boldsymbol{\ell}}_i = \text{norm}(\boldsymbol{\ell}_i \odot \boldsymbol{\nu}).$$

For $M = 1$, the mean is zero and both weight vectors are all ones. No query feature enters these fitted statistics.

For support rows r, t, define

$$Q_{rt} = \tfrac{1}{2}\left(\widetilde{\mathbf{h}}_{i_r}^\top \widetilde{\mathbf{h}}_{i_t} + \widetilde{\boldsymbol{\ell}}_{i_r}^\top \widetilde{\boldsymbol{\ell}}_{i_t}\right), \qquad K_{rt} = \exp\left(\frac{\min(Q_{rt} - 1, 0)}{h_K}\right).$$

Take distances $\max(1 - Q_{rt}, 0)$ over unordered off-diagonal pairs, retain those above 10^{-8}, and let d_{med} be their median (or 0.1 if none remain). The bandwidth is $h_K = 4\max(d_{\text{med}}, 10^{-5})$. The vector $\mathbf{k}_i \in \mathbb{R}^M$ uses this same transformation and bandwidth between query i and each support row. These operations correspond to `fit_geometry`, `fit_evidence`, and `predict_fitted` in `src/asrp/project_core.py`.

Beta mixture. For component $k \in \{1, 2\}$, let $\alpha_{sk} = \mu_s c_k$ and $\beta_{sk} = (1 - \mu_s)c_k$, with $(c_1, c_2) = (1, 8)$. The normalized posterior masses are

$$\widetilde{\omega}_{sk} = \pi_k \frac{\text{B}(\alpha_{sk} + S_s, \beta_{sk} + n_s - S_s)}{\text{B}(\alpha_{sk}, \beta_{sk})}, \qquad \omega_{sk} = \frac{\widetilde{\omega}_{sk}}{\sum_{k'=1}^{2} \widetilde{\omega}_{sk'}}, \qquad (\pi_1, \pi_2) = (0.3, 0.7).$$

Here B is the beta function. The implementation evaluates log-beta functions and normalizes by log-sum-exp. These weights give b_s in Section 4.3. If the current subject is absent from the support, `kernel_probe.py` returns μ_s without fitting a residual for that subject.

Centered ridge fit. Let $Z \in \{0,1\}^{M\times J}$ be the membership matrix for the J subjects present in the support, and $D = \mathrm{diag}(n_1, \ldots, n_J)$. Every diagonal count is positive. Let I_M be the $M \times M$ identity and $\mathbf{y} = (y_r)_{r\in\mathcal{D}_b} \in \mathbb{R}^M$. Define

$$H = I_M - ZD^{-1}Z^\top, \qquad K_c = HKH.$$

$$\gamma = H\left[\tfrac{1}{2}(K_c + K_c^\top) + \lambda_K I_M\right]^{-1} H\mathbf{y}, \qquad \lambda_K = 0.3.$$

The target $H\mathbf{y}$ subtracts each subject's empirical mean; the Beta mean b_s enters the final prediction separately. The code evaluates the inverse expression with a linear solve. Symmetrization matches the floating-point implementation; in exact arithmetic K_c is symmetric. The fitted subject contribution is

$$\bar{f}_s = \frac{1}{n_s}\sum_{r\in\mathcal{D}_s} (K\gamma)_r.$$

Substituting b_s, γ, $\mathbf{k}_i$, and $\bar{f}_s$ into Section 4.3 gives the implemented kernel prediction. At $n_s = 0$ this residual expression is bypassed; at $n = 0, 1$ the final ensemble weight w_n is zero.

A.4 Mixing and calibration for Section 4.4

The six released mixing weights are

n	0	1	3	7	15	31
η_n	0.2875	0.3000	0.2875	0.2750	0.2250	0.2375
w_n	0	0	0.2	0.4	1	1

Here η_n maps to `runtime_config.json:lambdas`; w_n is the active `PROBE_WEIGHTS` in `src/asrp/model.py`; and λ_K maps to `probe_config.json:ridge`. They are distinct parameters. The formulas in this report concern the official checkpoint counts; defensive interpolation/fallback behavior at other counts is not used in the experiment.

Let $t = t(i)$ be the first explicit topic key–value entry, in visible metadata order, present in the frozen topic table. If none matches, assign the Rasch embedding e_i to the closest of eight frozen centroids by cosine similarity and use its cluster entry. Write C_t^{cal} for that entry's frozen count and $r_{t,n}$ for its signed correction at budget n. Then

$$\Delta_n(i) = \mathrm{clip}\left(\frac{C_t^{\mathrm{cal}}}{C_t^{\mathrm{cal}} + a_t^{\mathrm{cal}}}\, r_{t,n}, -d_t^{\mathrm{cal}}, d_t^{\mathrm{cal}}\right), \qquad (a_t^{\mathrm{cal}}, d_t^{\mathrm{cal}}) = \begin{cases} (40, 0.06), & \text{explicit topic,} \\ (200, 0.03), & \text{cluster.} \end{cases}$$

If no entry exists, $\Delta_n(i) = 0$. Cluster cosine normalization floors norms at 10^{-8} and resolves ties by the first maximum. At the six checkpoints, the residual array index is the corresponding budget index.

Let $g^{\mathrm{stored}}(b)$ be the guard stored under the exact runtime benchmark identifier in the calibration artifact. The implemented guard is

$$g_n(b) = \begin{cases} \mathrm{clip}(g^{\mathrm{stored}}(b), 0, 0.35), & n \geq 15 \text{ and a key matches,} \\ 0, & \text{otherwise.} \end{cases}$$

It modifies w_n before mixing, while $\Delta_n(i)$ is added after mixing. A final clip to $[0,1]$ completes `predict`. The proxy's anonymous identifiers have no matching source keys.

A.5 Covariance calculation for Section 4.5

The acquisition kernel is denoted K^{acq} to distinguish it from the prediction kernel K. Its geometry is fitted to the concatenation of acquired support $\mathcal{D}_b$, the reference list $\mathcal{P}$, and the current candidate $*$. The same visible item may appear in both support and reference positions; these occurrences remain in the concatenation. Only $\mathcal{P}$ itself is deduplicated. Thus acquisition may use seen but unlabeled features, whereas prediction geometry uses only acquired support features.

Use the feature normalization and RBF rule in Appendix A.3, but compute the median-distance bandwidth from pairs in $\mathcal{P}$ only. Let $\mathcal{U}$ concatenate $\mathcal{P}$ and $*$. For the current subject s, augment the kernel blocks as

$$A_{\mathcal{D}\mathcal{D}}[r,t] = K^{\mathrm{acq}}_{\mathcal{D}\mathcal{D}}[r,t] + 0.25\,\mathbf{1}[s_r = s_t],$$
$$A_{\mathcal{U}\mathcal{D}}[j,r] = K^{\mathrm{acq}}_{\mathcal{U}\mathcal{D}}[j,r] + 0.25\,\mathbf{1}[s_r = s],$$
$$A_{\mathcal{U}\mathcal{U}} = K^{\mathrm{acq}}_{\mathcal{U}\mathcal{U}} + 0.25\mathbf{1}\mathbf{1}^\top,$$

where block subscript $\mathcal{D}$ abbreviates $\mathcal{D}_b$. Conditional covariance and gain are

$$\Sigma = A_{\mathcal{U}\mathcal{U}} - A_{\mathcal{U}\mathcal{D}}(A_{\mathcal{D}\mathcal{D}} + I_M)^{-1}A_{\mathcal{U}\mathcal{D}}^{\top}, \qquad G_j = \max\left(\frac{|\mathcal{P}|^{-1}\sum_{r\in\mathcal{P}}\Sigma_{rj}^2}{1+\max(\Sigma_{jj},0)}, 10^{-12}\right).$$

The identity term implements unit observation noise; without support, the subtraction is omitted. Define

$$\overline{G}_{\mathcal{P}} = |\mathcal{P}|^{-1}\sum_{r\in\mathcal{P}} G_r, \qquad V_j = \left[\min\left(\frac{G_j}{\max(\overline{G}_{\mathcal{P}}, 10^{-12})}, 20\right)\right]^4.$$

These utilities enter π_*^{acq} in Section 4.5. The power four is applied before dividing by mean reference utility. Gain depends on support membership and features; the numerical values of the revealed outcomes do not enter this covariance calculation.

Streaming boundaries and deterministic draw. Let A be the initial candidate count and $B_{\max} = 31$ the total label allowance in this experiment. Warm-up skips the first $W = \min(16, \max(0, A - B_{\max}))$ candidates. Checks occur in this order: reject if $L \le 0$; acquire if $R \le L$; otherwise skip during warm-up. After warm-up, an empty reference list falls back to probability L/R; otherwise use π_*^{acq}.

The code serializes `[context,input]` as sorted-key compact JSON with ASCII escaping, takes the first eight bytes of its SHA256 digest as an unsigned big-endian integer ζ, and sets $\xi = \zeta/2^{64}$. It acquires when $\xi < \pi_*^{\mathrm{acq}}$ (or $\xi < L/R$ in the fallback). This is a deterministic pseudorandom variate, not a new API call. The reference list uses the last 64 candidates before the current one, deduplicated by the hash of visible item text/metadata/interactor content. Skipped outcomes are never supplied. See `AdaptiveGainPolicy.choose` in `src/asrp/adaptive_gain_policy.py`.

B Public source inventory

The following 92 public binary source identifiers occur in the paired three-seed evaluation. They identify the evaluation sources; the benchmark inventory and upstream licenses remain authoritative.

```
afrieval
agentic_review_perturbation
algotune
alignment_faking
appworld
averimatec
babilong
beavertails
benger
bird_sql
braveguard
bridging_gap
cellverse
critic_discernment_game
crow
devbench
editbench
edu_circuit_hw
egoschema
elicitation_game
engdesign
enginemt_qa
exploitgym
faithcot
fewshot_ttt_bbh
financebench
finegrain_t2i
hallusionbench
harmbench
healthadminbench
helm_bbq
helm_thaiexam
hle
igakuqa119
ikp
jailbreakbench
kernelbench
```

```
legal_rag_bench
llmail_inject
llms_lean_formalization
mathconstraint
mathvista_mini
medhelm_ehr_sql
medhelm_head_qa
medhelm_med_mcqa
medhelm_med_qa
medhelm_medbullets
medhelm_medcalc_bench
medhelm_medec
medhelm_medhallu
medhelm_pubmed_qa
medhelm_race_based_med
mj_bench
mmbench_v11
monkey_power_laws
morphkv
mtbbench
multi_swebench
naep_llm_students
naturebench
nester
nyu_ctf_bench
openbiorq
os_harm
perfcodebench
planbench
proofaug
pxplore
real_webagents
realtimeqa
reliancescope
researchcodebench
revolve
risebench
robust_reasoning_benchmark
safeagentbench
sib200
software_world_models
sustainable_food
swe_smith
swebench_live
swepolybench
tau2_bench
taubench
truthfulqa_mc
tulu_human_eval
tumlu
visual_riddles
visualwebarena
vl_rewardbench
workarena
workbench_revisited
```

References

Frederic M. Lord. *Applications of Item Response Theory to Practical Testing Problems*. Lawrence Erlbaum Associates, 1980.

Sang Truong, Yuheng Tu, Percy Liang, Bo Li, and Sanmi Koyejo. Reliable and efficient amortized model-based evaluation. *arXiv preprint arXiv:2503.13335*, 2025. URL https://arxiv.org/abs/2503.13335.

Chris Ge, Daria Kryvosheieva, Daniel Fried, Uzay Girit, and Kaivalya Hariharan. Agent psychometrics: Task-level performance prediction in agentic coding benchmarks. In *Conference on Language Modeling*, 2026. URL https://arxiv.org/abs/2604.00594.

Sin-Han Yang, Cheng-Kuang Wu, Chieh-Yen Lin, Yun-Nung Chen, Hung-yi Lee, and Shao-Hua Sun. On calibration of large language models: From response to capability. *arXiv preprint arXiv:2602.13540*, 2026. URL https://arxiv.org/abs/2602.13540.

AIMS Foundations. measurement-db: Public benchmark inventory and response data. Hugging Face dataset, 2026. URL https://huggingface.co/datasets/aims-foundations/measurement-db.

Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems*, 2020. URL https://arxiv.org/abs/2002.10957.

Sang Truong, Wale Salaudeen, Nhi Truong, Serena Wang, Angelina Wang, Luke Guerdan, Ziang Xiao, Xing Xie, Nick Haber, and Sanmi Koyejo. PAIEC predictor baselines. GitHub repository, 2026. URL https://github.com/aims-foundations/paiec_baseline.